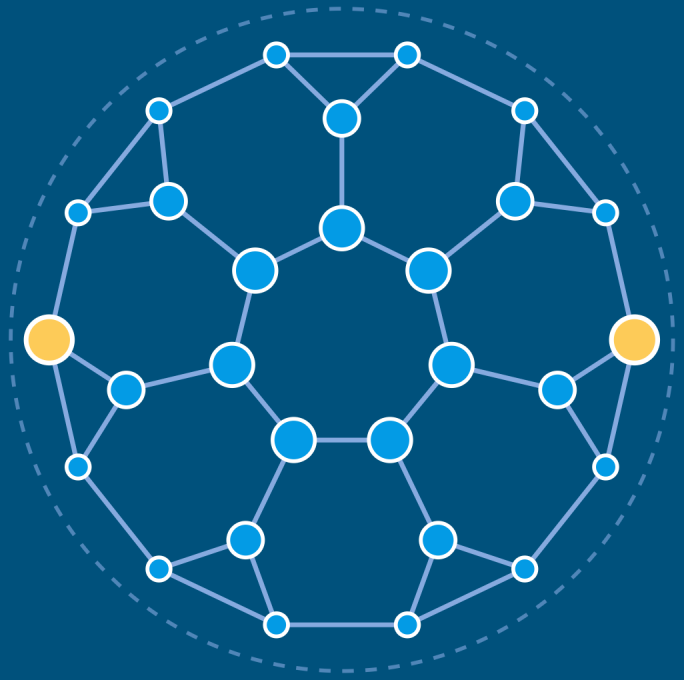
QISKIT FALL FEST 2026 · HACKATHON CHALLENGE

Geometry-Aware Quantum Cloud Challenge



Route a verified Bell pair across three processor architectures, then protect it.

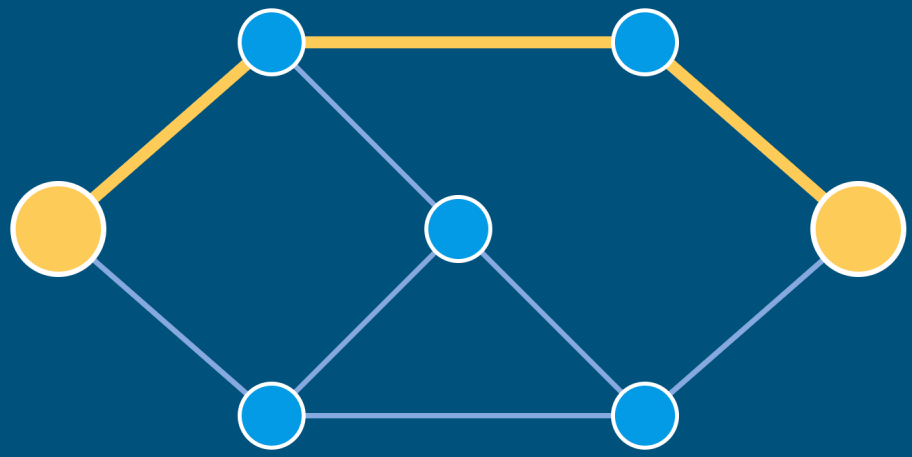
Hosted by the Quantum Student Organization at Purdue

Not Every Qubit Is Connected

Real chips link each qubit to only a few neighbors.

To make two distant qubits interact, the compiler inserts SWAP gates that walk them across the coupling graph.

Each SWAP costs three CNOTs, and every CNOT adds noise. The shape of the graph decides how much you pay.



Yellow: two designated ports, three hops apart.
Connecting them costs 2 SWAPs, or 6 extra CNOTs.

Why Geometry Matters

RIGHT HERE AT PURDUE

A campus quantum network

Purdue’s quantum network testbed links three labs in MSEE by fiber and shares entangled photons between them.

THE HARDWARE IDEA

Hyperbolic lattices

Finite hyperbolic lattices arrange connections and cycles differently from flat grids. What does that do to routing and error detection?

THE PLATFORM

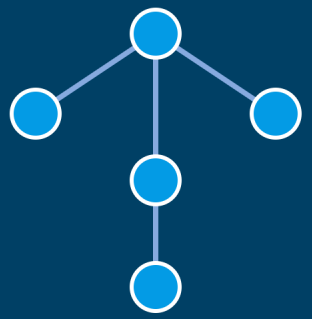
A decade of cloud access

IBM put a five-qubit processor on the cloud in 2016. Qiskit lets us compare past, present and hypothetical chips in one framework.

Three Processor Models

2016-Inspired

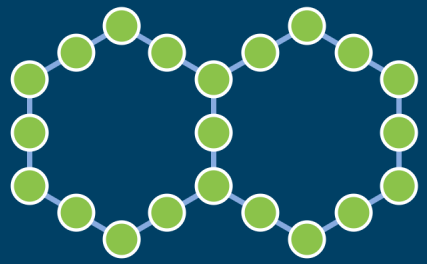
Five-qubit reference



Five-node graph
Synthetic noise model

IBM-Style Today

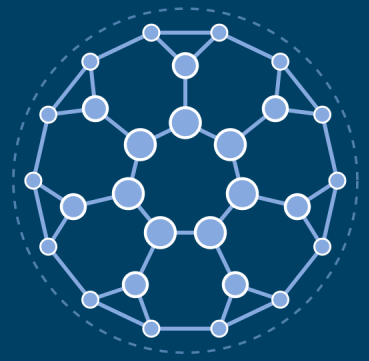
Backend snapshot or fake backend



Native gate set
Modern compilation baseline

Hyperbolic-Inspired

Lattice-derived coupling graph



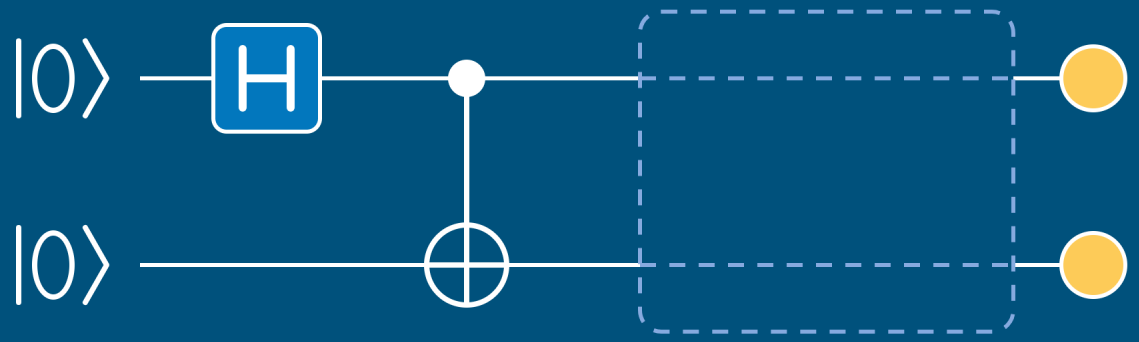
Hypothetical connectivity
Circuit-level noise only

Sketches are illustrative. Use the official graphs, ports and noise models from the starter kit.

The Core Task: A Bell Pair

Goal: create $|\Phi^+\rangle = (|00\rangle + |11\rangle) / \sqrt{2}$ between the two ports.

> **Port:** a fixed physical qubit, two per chip, listed in the starter kit. Your pair must finish on them, so the distance between ports sets the routing cost.



Prepare with H and CNOT, then compile and route (dashed) so the pair lands on the two ports (yellow).

VERIFY IN THREE BASES

XX · YY · ZZ

Matching Z results alone could come from a classical coin flip. XX and YY show the pair is truly entangled.

Verifying the Bell Pair

Step 1: Measure three ways

Run the same circuit three times. Only the last gates change:

ZZ — measure as is

XX — H on both qubits first

YY — S†, then H on both first

Step 2: Score each one

For each run, take the fraction of shots where the two bits agree, minus the fraction where they disagree.

A perfect pair scores:

ZZ = +1, XX = +1, YY = −1

Step 3: Combine the scores

$$F = [1 + XX - YY + ZZ] / 4$$

Perfect pair: F = 1.

Above 0.5: definitely entangled.

Example: XX = 0.90, YY = −0.90, ZZ = 0.95 gives F ≈ 0.94

Five Levels, One Benchmark

Each level builds on the last. Finish Levels 1–3 and you have a complete submission.

1

Make it

An ideal Bell pair on all three graphs

EXPLORER

2

Break it

Add noise and measure the damage

EXPLORER

3

Protect it

Detect errors with one parity check

EXPLORER

4

Optimize it

Smarter layout, routing and checks

BUILDER

5

Rethink it

Encodings shaped by the geometry

RESEARCH

CORE BENCHMARK · REQUIRED FOR EVERY TEAM

EXTENSIONS · GO FURTHER

LEVEL 1 Explorer · Required · Cases 1, 4, 7

Make It: An Ideal Bell Pair

YOUR TASK

1. Build the Bell circuit: H on one qubit, then a CNOT.
2. Load each coupling graph and pin your qubits to its two ports.
3. Transpile, then record depth, two-qubit gates and SWAPs.
4. Measure ZZ, XX and YY on a noiseless simulator and compute F.

QISKIT TOOLS

```
CouplingMap(edges)
transpile(qc, coupling_map=cm,
  initial_layout=[a, b])
AerSimulator()
```

WATCH OUT

Routing can move qubits. Check the final layout so the pair really ends on the ports.

LEVEL 2 Explorer · Required · Cases 2, 5, 8

Break It: Add Realistic Noise

YOUR TASK

1. Attach each model's noise, using the common noise assumptions.
2. Rerun the same transpiled circuits from Level 1.
3. Report F with error bars: use enough shots, then bootstrap.
4. Plot fidelity against two-qubit gate count and SWAPs.

NOISE PER MODEL

2016 → synthetic model
IBM-style → fake backend
Hyperbolic → circuit-level

THINK ABOUT

Is the drop caused by extra SWAPs, or by different gate errors?

LEVEL 3 Explorer · Required · Cases 3, 6, 9

Protect It: Detect Errors

YOUR TASK

1. Add one justified check, e.g. measure the ZZ parity onto an ancilla.
2. For $|\Phi^+\rangle$ that parity is +1, so a −1 flags a bit flip: discard the shot.
3. Report conditional fidelity, acceptance fraction and the extra cost.
4. Compare with your Level 2 baseline on all three models.

THINK ABOUT

Where can the ancilla sit on each graph? If it needs routing, it brings its own noise.

THE RULE

Higher fidelity on fewer shots is half the story. Always report the yield.

Checkpoint: The Core Benchmark

Levels 1–3 fill these nine cells. Every team completes all nine before going further.

Processor model	Level 1 · Ideal	Level 2 · Noisy	Level 3 · Protected
2016-inspired	Case 1	Case 2	Case 3
IBM-style today	Case 4	Case 5	Case 6
Hyperbolic-inspired	Case 7	Case 8	Case 9

For each model, compare the noisy baseline with the protected result and what the protection cost.

LEVEL 4 Builder · Extension

Optimize It: Route Smarter

YOUR TASK

1. Compare layouts and routing methods, e.g. SABRE vs. a hand-picked path.
2. Route entanglement, not gates: join Bell pairs on neighboring links by entanglement swapping.
3. Upgrade your check: ZZ and XX together, or ancillas on short cycles.
4. Weigh each gain against the extra operations it costs.

QISKIT TOOLS

```
transpile(qc, coupling_map=cm,
  routing_method="sabre",
  seed_transpiler=7)
with qc.if_test((c, 1)):
  qc.x(q)
```

KEY QUESTION

Does the fidelity gain beat the cost of the extra operations?

LEVEL 5 Research · Extension

Rethink It: Use the Geometry

YOUR TASK

1. Fit an encoding or stabilizer network to the graph's own cycles.
2. Measure its overhead: qubits, depth, two-qubit gates and yield.
3. Sweep noise strength, port distance or graph size to study scaling.
4. Optional: other noise models, bigger circuits or real QPU runs, reported separately.

KEY QUESTION

How does the effect scale with graph size and noise strength?

THE CAVEAT

One finite model never proves a universal advantage. Say what your result does and doesn't show.

What Every Team Reports

Measure	Report
Output quality	Bell fidelity with uncertainty
Circuit cost	Depth and number of two-qubit gates
Routing	SWAP overhead
Qubit use	Physical qubits, including ancillas
Postselection	Acceptance fraction and discarded shots

The same five numbers for every case, so results compare directly across models and teams.

Interpreting Your Results

NOISE AND COMPILATION

Explain the drop

Tie every change in fidelity to the operations compilation added and to the noise model you assumed.

PROTECTION OVERHEAD

Was it worth it?

Does your check make up for its extra gates, ancillas and discarded shots?

ARCHITECTURE

Separate the causes

Split connectivity effects from gate-set or noise differences. One finite model doesn’t prove a universal advantage.

Ground Rules

Everyone does

- Use the common processor definitions, fixed ports and noise assumptions.
- Complete the three-model comparison and test one justified protection method.
- Report reproducible results, including added operations and postselection yield.

You choose

- Your layout, routing method and protection strategy.
- Extras like alternative noise models, larger circuits or geometry-aware encodings.
- Keep optional studies and QPU results separate from the common benchmark.

What to Submit

01	**Reproducible notebook**	Qiskit code, assumptions and recorded results
02	**Graphs and circuits**	The three coupling graphs with ports, plus the transpiled circuits
03	**Quantitative comparison**	Fidelity, circuit cost and yield across all nine cases
04	**Scientific interpretation**	What your results show, and where they stop
05	**Hardware or software recommendation**	One design choice for future cloud systems, backed by your results

How You'll Be Judged

Technical submission · 60%

Scientific correctness and reproducibility	18%
Quality of the architecture comparison	12%
Protection or error-detection strategy	12%
Effective use of Qiskit	9%
Evidence for the future-system recommendation	6%
Clarity of the written submission	3%

Live presentation · 40%

Explaining your approach and results	15%
Answering the judges' questions	10%
Slides and visuals	10%
Staying on time	5%

Let’s Build

SATURDAY
8:00 PM
Hacking begins

SUNDAY
4:00 PM
Hacking ends, submit your work

SUNDAY
4:00–5:30
Presentations and judging

SUNDAY
6:30 PM
Results and closing remarks

Starter kit and processor definitions: Link to Repo

Start at Level 1, ask questions, and have fun :)

Scan to join the Discord